

2025 年 CCF 大学生计算机系统与程序设计竞赛

CCF CCSP 2025

时间：2025 年 10 月 22 日 09:00 ~ 21:00

题目名称	仓库价格调整	堆上内存分配	云函数序列	数据库查询优化	芯片系统设计优化
题目类型	传统型	传统型	传统型	传统型	传统型
输入	标准输入	标准输入	标准输入	标准输入	标准输入
输出	标准输出	标准输出	标准输出	标准输出	标准输出
每个测试点时限	1.0 秒	1.0 秒	1.0 秒	1.0 秒	1.0 秒
内存限制	512 MiB	512 MiB	512 MiB	512 MiB	512 MiB
子任务数目	10	20	20	10	10
测试点是否等分	是	是	是	是	是

仓库价格调整 (XOR)

【题目背景】

小新是一名全栈工程师，刚加入全球智能仓储公司 SmartDepot。这家公司运营着一个集仓储管理、云端计算、数据库服务与硬件加速于一体的智能系统，负责全球范围内的物流调度与计算任务。为了确保 SmartDepot 的整个业务链条高效运转，小新决定寻求你的帮助，一起完成一系列关键任务。

【题目描述】

公司仓库中，依次摆放着一排共有 m 件货物（编号从 1 到 m ，表示它们的固定顺序）。每件货物都有一个互不相同的整数价格 A_i 。

由于系统升级时出了一点小差错，部分货物在盘点时出现价格顺序混乱，因此需要小新对所有货物的价格进行调整。具体做法是选择一个整数 X ，并将每个货物的价格调整为

$$B_i = A_i \oplus X$$

其中 $\oplus$ 表示按位异或。

如果某件货物在位置上靠前，但其价格比后面货物的价格还高，就会造成混乱。小新希望选择合适的掩码 X ，使得在调整后的混乱程度更小。

更准确地说，若一对货物 (i, j) 满足 $i < j$ 且 $B_i < B_j$ ，则称它们为一个“合法对”。你的任务是：

- 找到最大化合法对数量的最优掩码 X ，
- 输出最大合法对数量和最小的最优掩码 X 。

【输入格式】

从标准输入读入数据。

输入的第一行包含一个正整数 m 。

输入的第二行包含 m 个正整数 $A_1, A_2, \dots, A_m$ ，表示每件货物的原本价格。保证所有 A_i 两两不同，且 $0 \leq A_i \leq 2^{30}$ 。

【输出格式】

输出到标准输出。

输出的第一行为一个整数，表示最大合法对数量。输出的第二行为一个整数，表示最小的最优掩码 $X_{\min}$ ，也就是所有能达到最大合法对数量的掩码中，数值最小的那个。

【样例输入】

```
1 5
2 12 15 6 0 4
```

【样例输出】

```
1 8
2 8
```

【样例解释】

取 $X = 8$ 时, $B = [12 \oplus 8, 15 \oplus 8, 6 \oplus 8, 0 \oplus 8, 4 \oplus 8] = [4, 7, 14, 8, 12]$.

满足 $i < j$ 且 $B_i < B_j$ 的合法对共有 8 个, 分别是

$(1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (4, 5)$

【子任务】

- 在 30% 的测试中, $m \leq 2,000$, 且最终的 X 小于 1000。
- 在 50% 的测试中, $m \leq 2,000$
- 在 100% 的测试中, $m \leq 200,000$

堆上内存分配 (Heap)

【题目背景】

现代应用编程语言通常使用垃圾回收 (Garbage Collection, GC) 支持堆内存回收, 回收的堆内存可用于后续的内存分配。堆内存的使用就是一个内存“分配-回收-分配”的循环。支持内存自动分配和回收的堆称为“受管堆 (Managed Heap)”。受管堆内存回收过程中, 活对象标记是不可或缺的环节, 各类 GC 都是基于活对象标记算法识别堆中的存活对象, 据此对堆内存进行相应处理完成内存回收重用。活对象标记过程是从给定的根指针集合出发, 递归遍历根指针直接或者间接指向的所有对象, 被遍历到的对象就是活对象, 反之就是死对象, 死对象占据的内存可被回收用于后续内存分配。

【题目描述】

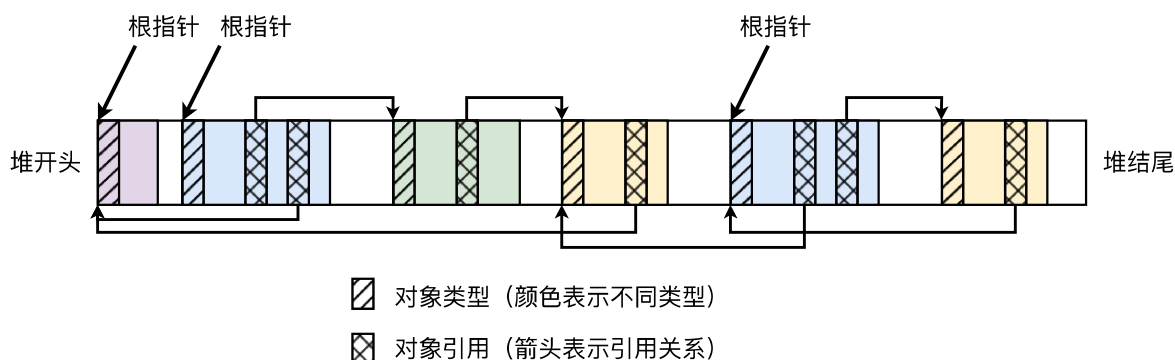
在本题中, 受管堆是如下定义的一个数据结构:

1. 包含一段地址连续的内存。
2. 内存中储存有一至多个对象, 分布在内存空间中, 对象占用的内存彼此不重叠。
3. 对象中依次存放着 1 个类型 ID, 若干 (大于或等于 0) 个引用字段, 其值是某个对象的地址; 其余字段为值。类型 ID 用于指明该对象的大小, 以及该对象的引用成员所在的偏移位置。
4. 根指针集合中的元素是指向堆中对象的指针。

以根指针集合中的元素作为遍历起点, 堆中所有可遍历对象构成了一个带有根节点的有向图, 活对象是其中的节点, 活对象的引用成员是指向其它活对象的边。遍历该有向图的过程称为“标记 (Mark)”, 可识别出堆中所有的活对象。

堆文件及其格式

本题设定受管堆内存具有下图所示的结构:



对象格式

堆数据文件是堆内存的转储文件, 以二进制数据形式提供, 它包含了对对象的二进制数据。对象在内存中的数据结构定义如下 (假设指针为 64 位宽度, 对象成员地址按照 8 字节对齐):

```
1 struct Object {  
2     TypeId tid; // 类型的 ID  
3     uint64_t val[]; // 引用成员和值成员构成的数组  
4 };
```

对象包含类型信息、值字段和引用字段。如下为一个对象的示例：{5, 200, 400, 0x1230, 0x5678, 100, 300}。其中的 5 表示类型的 ID。后续数据的解释需要参照具体的类型信息。假如 ID 为 5 的类型为 {56, 24, 32}，即该类型对象的大小为 56，包含两个引用字段，偏移量分别为 24 和 32 字节，对应第 3 和 4 个成员，即 0x1230、0x5678。其余成员（200、400、100、300）均为值类型，不会引用其他对象。注意，这里的大小是包含对象头部的“类型 ID”的，引用字段的偏移量同样是包含头部的“类型 ID”，因而引用字段的偏移量一定不为 0。

你需要完成以下两个任务：

任务一：标记活对象

分析给定的堆镜像文件，根据堆信息中指定的根指针遍历堆中活对象，统计活对象的数量和所占内存大小。

任务二：完成内存分配

在上述给定的堆中完成一组新的对象内存分配请求。

活对象正在占据的内存不可用于分配，其余所有内存均为“空闲内存”，可用于分配新对象。分配内存是从空闲内存中找到一份连续的内存空间且其大小超过给定的正整数。

- 如果连续的“空闲内存”足够大可直接用于分配指定大小，则分配开销计为 0。
- 如果“空闲内存”不够大，则需要先选择并迁移一些活对象到外部内存，形成更大的连续空闲内存块，再进行分配。为简化问题，假定外部内存不在堆中，且空间无限大。

迁移对象的开销包括两个部分：

1. 对象本身的大小（以字节为单位）；
2. 对象迁移到外部内存后，需要更新所有指向该对象的指针，包括根指针和活对象（也包括外部内存上的对象）中存储的旧指针，其开销为 需要修改的指针个数*8。

这组分配请求由 N 个整数 R_i 组成，依次表示对象所需的内存大小。这组对象的分配需要满足以下两条要求：

- 第 i 个对象分配到的地址 A_i ，要小于第 $i+1$ 个对象分配得到的地址 A_{i+1} ，即对于 $i < j$ ，需要满足 $A_i < A_j$ 。
- 这一组新的对象必须位于堆内存，不能被迁移到外部内存。

你需要计算出满足这组对象分配所需要的最小开销。

【输入格式】

从标准输入读入数据。

标准输入的第一行包含五个整数：堆的起始地址 A （十六进制），堆的大小 S 、对象类型数 T 、根指针数量 P 、新的对象分配请求数量 N 。

之后 T 行，依次给出 ID 为 1 到 T 的类型描述。对象类型描述为一组空格隔开的整数，第一个整数为该类型对象占内存的大小，之后的若干个整数表示该对象中各个引用成员的偏移量。

之后 P 行，每行表示一个根指针（十六进制）。

之后 N 行，每行一个整数 R_i 表示需要分配的对象的大小。注意，所有分配的对象地址需要 64 位对齐。

再之后的若干行，每行一对十六进制数字（空格隔开），分别表示一个 8 对齐的内存地址和该内存地址上的 64 位数据，这部分数据不需要模拟字节序，直接按给出的数据读取即可。

【输出格式】

输出到标准输出。

输出两行。第一行为两个整数，分别代表活对象的数量和所占内存的大小。

第二行为一个整数，表示完成所有新对象分配所需的最小开销。如果无法完成分配，输出 -1。

【样例】

见题目目录下的 *1.in* 与 *1.ans*。

【子任务】

- 在前 10% 的测试中，活对象总数量不超过 10。
- 在前 40% 的测试中，活对象总数量不超过 30，请求数量不超过 5，堆的大小不超过 10^6 。
- 在前 50% 的测试中，活对象总数量不超过 40。
- 在前 100% 的测试中，活对象总数量不超过 200，请求数量不超过 200，堆的大小不超过 10^7 。

云函数序列 (order)

【题目描述】

公司的云端计算平台采用 Serverless 架构。服务器无感知计算 (Serverless Computing) 是一种主流云计算范式, 其核心特征是开发者无需管理底层服务器, 由云平台根据实际需求动态分配资源并运行代码。开发者只需编写函数 (Function), 云平台会自动负责函数的部署、调度与伸缩。现在有一个函数到达的序列, $f_1, f_2, \dots, f_n$, 每个函数有一个属性值 a_i 。每个函数的属性值互不相同。作为平台需要根据资源合理的排列这些函数, 因此涉及到函数的重新编排。重新编排函数是复杂的, 为了简单理解, 本题中重新编排的操作简化为在原序列中选择一系列不相交的区间 $[L_i, R_i]$ 并且将每一个区间各自反转, 其代价为 $\sum R_i - \sum L_i$ 。显然, 满足代价不超过 c 的操作后序列有很多个。我们希望进行 Q 次查询, 查询按照字典顺序排序不重复的第 i 个序列的第 j 个 Serverless 函数的属性数值。特别地, 这里的序列编号指的是在符合本题条件中的所有序列的编号, 并非在全排列中的序号。

【输入格式】

从标准输入读入数据。

本题包含多组测试数据, 第一行输入一个整数 T 代表测试的组数。

在每组内, 第一行包括三个整数 N, c, Q 。分别代表 Serverless 函数的个数, 操作代价上限和总查询次数。接下来一行包含 N 个数, 为 Serverless 函数的属性值。随后紧接 Q 行查询, 每一行有两个整数 i, j 描述查询。

【输出格式】

输出到标准输出。

对于每一个查询请求, 输出查询的结果。如果不存在第 i 个序列, 输出-1。

【样例 1 输入】

```
1 1
2 3 1 4
3 1 2 3
4 1 1
5 2 2
6 3 3
7 59 1
```

【样例 1 输出】

```
1 1
2 3
3 3
4 -1
```

【样例 1 解释】

在样例中，有 3 个 Serverless 函数。满足交换数值不超过 1 的合法排序只有 1, 2, 3、1, 3, 2、2, 1, 3。因此，查询第一个序列的第一个数为 1，查询第二个序列的第 2 个数为 3，第三个序列的第三个数为 3。第 59 个序列不存在，输出-1。

【评分方式】

对于 20% 的数据，保证 $\sum N \leq 10, 1 \leq c \leq 2, \sum Q \leq 10$ 。

对于 50% 的数据，保证 $\sum N \leq 1000, 1 \leq c \leq 2, \sum Q \leq 1000$ 。

对于 100% 的数据，保证 $\sum N \leq 5 \times 10^5, 1 \leq c \leq 5, \sum Q \leq 5 \times 10^5$ 。

数据库查询优化 (Query)

【题目背景】

在现代软件应用中，数据库扮演着至关重要的角色，它们如同海量数据的“中央图书馆”。为了从这个“图书馆”中高效地检索信息，我们使用一种名为 SQL (Structured Query Language) 的“查询语言”。然而，对于同一个查询需求，数据库内部可以有多种执行方式，这些方式的效率天差地别，就像从图书馆找一本书，你可以逐层逐个书架地毯式搜索，也可以先通过索引系统定位到书架再寻找。一个优秀的数据库，其核心组件之一就是**查询优化器**，它的任务是在所有可能的执行路径中，选择成本最低、效率最高的一条。

本题目需要你扮演查询优化器的角色。我们先通过一个具体的例子来理解数据库是如何工作的。

在本题目中，我们将使用一种我们称之为 BaseSQL 的简化类 SQL 语言。

<BaseSQL 定义>

1. **核心结构**: 一个 BaseSQL 查询是一个**选择-投影-连接** (Select-Project-Join, SPJ) 查询。它包含 SELECT、FROM 和 WHERE 子句。

```
1 SELECT T1.C1, T1.C2, T2.C1 AS C
2 FROM T1, T2, ...
3 WHERE condition1 AND condition2 AND ...
```

在 SELECT 子句中，投影列可以是以下形式之一：

- *：表示选择所有列；
- 带表名前缀的列，如 T1.C2，则在结果表中，该列名字为 C2；
- 带显式别名的列，如 T1.C2 AS ColAlias 或 T1.C2 ColAlias（即 AS 可省略，二者语义完全相同），则在结果表中，该列名字为 ColAlias。

2. **表 (T_i)**: 在 FROM 子句中的“表”可以是以下两种之一：

- 一个数据库中的**基表** (如 Students)。
- 一个**子查询**，也就是另一个嵌套在括号内并赋予别名的 BaseSQL 查询。子查询可以包含投影列的显式别名；若使用 SELECT * 则继承底表或子查询的列名。子查询必须有别名；同一作用域内不允许出现同名列。示例：

```
1 (SELECT Students.name AS student_name FROM Students WHERE
   Students.age = 21) AS SeniorStudents
```

这个子查询会首先被执行，其结果被视为外部查询的一个临时表。

- 表或子查询的别名可通过空格直接声明，如 FROM Students S，也可以使用 AS 关键字声明，如 FROM Students AS S；两种写法在本题中完全等价。引入别名

后，后续条件中引用该表需使用别名（例如 `S.student_id`），这与引用原表名具有相同语义。

3. 条件: 在 `WHERE` 子句中的“条件”是一个等值连接，意味着它只使用 `=` 操作符。它可以是：

- 两个表之间的连接条件: `T1.columnA = T2.columnB`。
- 与一个整数常量值比较的过滤条件: `T1.columnC = 123`。

4. 子链接 (Sublink): 条件还可以是一个子链接，即一个列的值与一个聚合子查询的结果进行比较（列名一定在等号左边）。我们唯一支持的聚合是 `COUNT(*)`。需要注意的是，子链接的 `WHERE` 子句中最多只能有一个与外部查询相关的条件。

- 相关子链接: 子链接的 `WHERE` 子句引用了外部查询表中的一个列。这意味着对于外部查询的每一行，子链接都必须被重新求值。

```
1  -- 对于每一门课程，子链接计算该特定课程的报名人数。  
2  Courses.credits = (SELECT COUNT(*) FROM Enrollments E2 WHERE  
                      E2.course_id = Courses.course_id)
```

- 非相关子链接: 子链接的 `WHERE` 子句不引用任何外部的表。它可以被计算一次，其结果可以被复用。

```
1  -- 子链接一次性计算出课程 '101' 的总报名人数。  
2  T1.c1 = (SELECT COUNT(*) FROM Enrollments WHERE  
           Enrollments.course_id = 101)
```

- 相关性检测和处理:

- 相关子链接: 子链接的 `WHERE` 子句中包含对外部查询表的直接列引用
- 非相关子链接: 子链接完全独立，不依赖任何外部查询的数据
- 相关性判断规则: 检查子链接条件中引用的表是否属于当前查询作用域外的表
- 示例:

```
1  WHERE T1.c1 = (SELECT COUNT(*) FROM T2 WHERE T2.c2 =  
                  T1.c3)
```

* `T1.c3` 是外部引用，因此这是一个相关子链接

* 对于 `T1` 的每一行，都要用该行的 `T1.c3` 值重新计算子链接

- 重要约束 - 相关子链接的引用限制:

- 约束规则: 在相关子链接中，子查询内部只能引用与子链接条件左侧相同的外部表。同时，该引用只能出现在子查询 `WHERE` 子句中，不会出现在 `FROM` 中的子查询中。

– 正确示例:

```
1 SELECT * FROM T1
2 WHERE T1.col = (
3     SELECT COUNT(*)
4     FROM T2
5     WHERE T2.x = T1.y)
```

* 子链接条件左侧是 T1.col, 子查询内部引用的是 T1.y, 满足约束

– 错误示例:

```
1 SELECT * FROM T1, T3
2 WHERE T1.col = (
3     SELECT COUNT(*)
4     FROM T2
5     WHERE T2.x = T3.y)
```

* 子链接条件左侧是 T1.col, 但子查询内部引用的是 T3.y, 违反约束

– 错误示例:

```
1 SELECT * FROM T1, T3
2 WHERE T1.col = (
3     SELECT COUNT(*)
4     FROM T2, (SELECT * FROM T3 WHERE T1.x = T3.y))
```

* 子链接条件左侧是 T1.col, 但子查询内部引用的 T1.x 在 FROM 中, 违反约束

* 多表连接中的应用: 在 SELECT * FROM T1, T2, T3 的查询中, 如果有子链接 T1.a = (SELECT COUNT(*) FROM T4 WHERE ...), 则子查询内部的相关条件只能引用 T1 的列, 不能引用 T2 或 T3 的列

5. 列引用规范: 所有列引用必须以“表名或子查询别名. 列名”的形式书写; 不允许未限定列名。例如: Students.age = 21, Enrollments.course_id = Courses.course_id。

< 实例中的 BaseSQL >

下面让我们通过一个例子来继续介绍 BaseSQL。

1. 基础数据表

假设我们的大学数据库中有三张核心表:

- **Students (学生表)**: 存储学生信息。

student_id	name	age
1	Alice	21
2	Bob	20
3	Charlie	21
4	David	19

- **Courses** (课程表): 存储课程信息。

course_id	course_name	credits
101	Math	2
102	Physics	3
103	History	2

- **Enrollments** (选课表): 记录学生选修了哪些课程。

student_id	course_id
1	101
	103
2	102
3	101
	102
	103

2. 一个复杂的查询请求

现在，我们想提出一个有点复杂的需求：“找出所有年龄等于 21 岁的学生，他们所选修的课程中，有哪些课程的学分（credits）恰好等于选修该门课程的总人数？”

这个需求转换成我们的baseSQL，是这样的：

```
1 SELECT *
2 FROM
3     (SELECT
4         Students.student_id,
5         Students.name AS student_name,
6         Students.age
7     FROM Students
8     WHERE Students.age = 21
9     ) AS SeniorStudents,
10    Enrollments,
11    Courses
12 WHERE
```

```

13     SeniorStudents.student_id = Enrollments.student_id
14 AND
15     Enrollments.course_id = Courses.course_id
16 AND
17     Courses.credits = (SELECT COUNT(*)
18                         FROM Enrollments E2
19                         WHERE E2.course_id = Courses.course_id);

```

这个查询包含了子查询、子链接和连接：

- **子查询 (Subquery)**: FROM 后面的 (SELECT * FROM Students WHERE age > 21) AS SeniorStudents 就是一个子查询。数据库会先执行这个查询，生成一个只包含高年级学生的临时表 SeniorStudents，然后在后续操作中使用它。
- **连接 (Join)**: FROM 子句中列出了三个表 (SeniorStudents、Enrollments、Courses)，WHERE 子句中的 SeniorStudents.student_id = Enrollments.student_id 和 Enrollments.course_id = Courses.course_id 则是连接条件，用于将这三张表的数据按照正确的对应关系拼接起来。
- **相关子链接 (Correlated Sublink)**: WHERE 子句中最后的条件 Courses.credits = (SELECT COUNT(*) ...) 是一个典型的相关子链接。它的特殊之处在于，内部的子查询 (SELECT COUNT(*) ...) 并非只计算一次，而是“相关”于外部的查询。对于外部查询处理的每一行数据，特别是每一行的 Courses.course_id，它都会被传入内部子查询中，重新计算一次选课人数。

3. 执行计划与执行过程

数据库有多种方式来执行上述查询，一个简单的执行计划可能是 Join(Join(SeniorStudents, Enrollments), Courses)。我们来看看它是如何一步步得到结果的。

Step 1: 执行子查询

首先，执行

```

1 (SELECT
2     Students.student_id,
3     Students.name AS student_name,
4     Students.age
5 FROM Students
6 WHERE Students.age = 21)

```

得到临时表 SeniorStudents，其中既保留列 student_id、age，也把 name 重命名为 student_name。

student_id	name	age
1	Alice	21
3	Carol	

Step 2: 连接 SeniorStudents 和 Enrollments

将上一步的结果与 Enrollments 表根据

`SeniorStudents.student_id = Enrollments.student_id` 进行连接。

这个过程可以想象成一个嵌套循环：

```

1 // 伪代码
2 for rowA in SeniorStudents:
3     for rowB in Enrollments:
4         if SeniorStudents.student_id == Enrollments.student_id:
5             output(rowA, rowB)

```

得到中间结果 Temp1, 由于纸张宽度限制, SeniorStudents 缩写为 SS, Enrollments 缩写为 En, Courses 缩写为 Co, 下同。Temp1 为：

SS.student_id	SS.name	SS.age	En.course_id
1	Alice	21	101
			103
3	Carol		101
			102
			103

Step 3: 连接 Temp1 和 Courses

将上一步的结果 Temp1 与 Courses 表根据 `Enrollments.course_id = Courses.course_id` 连接，得到完全连接后的结果 Temp2。

SS.student_id	SS.name	SS.age	En.course_id	Co.course_id	Co.course_name	Co.credits
1	Alice	21	101	101	Math	2
			103	103	History	
3	Carol		101	101	Math	3
			102	102	Physics	
			103	103	History	2

Step 4: 应用相关子链接进行过滤

现在，对 Temp2 的每一行执行过滤条件 `Courses.credits = (SELECT COUNT(*) FROM Enrollments E2 WHERE E2.course_id = Courses.course_id)`。

- 对于第一行 (Math): Courses.credits 是 2。执行 `SELECT COUNT(*) FROM Enrollments WHERE Enrollments.course_id = 101`, 结果是 2 (Alice 和 Carol 都选了)。2 == 2 为真，保留该行。

- 对于第二行 (History): `Courses.credits` 是 2。执行 `SELECT COUNT(*) FROM Enrollments WHERE Enrollments.course_id = 103`, 结果是 2 (Alice 和 Carol 都选了)。 `2 == 2` 为真, 保留该行。
- 对于第三行 (Math): 与第一行逻辑相同, `2 == 2` 为真, 保留。
- 对于第四行 (Physics): `Courses.credits` 是 3。执行 `SELECT COUNT(*) FROM Enrollments WHERE Enrollments.course_id = 102`, 结果是 2 (Bob 和 Carol 选了)。 `3 == 2` 为假, 丢弃该行。
- 对于第五行 (History): 与第二行逻辑相同, `2 == 2` 为真, 保留。

Step 5: 最终结果

经过过滤后, 最终的查询结果为:

SS.student_id	SS.name	SS.age	En.course_id	Co.course_id	Co.course_name	Co.credits
1	Alice	21	101	101	Math	2
			103	103	History	
3	Carol		101	101	Math	3
			102	102	Physics	
			103	103	History	

这个例子展示了数据库执行查询的复杂性。不同的连接顺序 (比如先连接 `Courses` 和 `Enrollments`) 会产生不同的中间结果大小和计算成本, 而你的任务就是找到那个“最优”的顺序和方法。

【题目描述】

在本题中, 你需要读取并解析一个简化版的 SQL 查询 (`baseSQL`), 并为该查询找出一个最优的执行计划。执行计划决定了表的连接顺序和连接算法, 你的目标是使整个查询的总执行代价 (`Cost`) 最小。

核心概念定义

- 表 (Table): 数据库中的基础数据单元, 我们用 T_i 表示第 i 张表。
- 列 (Column): 表中的一个字段, 用 $T_i.C_j$ 表示表 T_i 的第 j 列。
- 行数 (Rows): 一个表或一个中间结果中包含的数据行数, 表示为 $Rows(T)$ 。
- NDV (Number of Distinct Values): 某一列中不重复值的数量, 表示为 $NDV(T.C)$ 。例如, `Enrollments` 表的 `course_id` 列值为 (101, 103, 102, 101, 102, 103), 其 NDV 为 3。

执行计划:

一个执行计划是一棵二叉树，其叶子节点是表，内部节点是连接操作。我们支持两种连接操作：**哈希连接 (Hash Join)** 和 **嵌套循环连接 (Nested Loop Join)**。计划由一个嵌套表达式表示，例如 $H(L(T_1, T_2), T_3)$ 。

计划空间枚举要求：

- **完整枚举：**必须枚举所有可能的连接顺序，包括左深树、右深树和 bushy 树结构
- **算法选择：**对每个连接操作，必须同时考虑哈希连接和嵌套循环连接两种算法
- **最优计划：**在所有可能的计划中选择总代价最小的方案

代价模型：

查询的执行过程被分解为一系列的**过滤 (Filter)** 和 **连接 (Join)** 操作，每个操作都会产生新的中间结果（其行数需要估算），并带来一定的执行代价。代价的计算需要进行“行数估算”和“操作代价”计算。

1. 行数估算 (Cardinality Estimation)

过滤 (Filter): 对一个表 T_i 应用过滤条件 $T_i.C_j = constant$ ，过滤后的期望行数为：
 $Rows(FilteredT_i) = Rows(T_i) / NDV(T_i.C_j)$

- **解释：**这假设了数据是均匀分布的。如果一个列有 100 个不同的值，那么筛选其中任何一个特定值的记录，期望会得到总行数的 1/100。
- **多个等值过滤的累积效应：**当同一表存在多个等值过滤条件（包括常量等值与子链接等值）时，对行数按各自列的 NDV 依次独立缩减；若多次作用于同一列，则按该列的 NDV 再次缩减。例如： $Rows = Rows / NDV(c_1) / NDV(c_2) / \dots$ ；若同列两次等值，则 $Rows = Rows / NDV(c) / NDV(c)$ 。
- 这种计算方法也适用于子链接过滤。因为在子语句执行完之后，等号右边的值已经确定，子链接会变化为 $T_i.C_j = constant$ 的形式。

连接 (Join): 对两个节点 A 和 B（A 和 B 自身可以是基础表或已连接的中间结果）进行连接，连接条件为 $A.C_a = B.C_b$ 。

- **选择率 (Selectivity):** 首先，定义单个连接条件的“选择率”，它表示数据能通过该条件过滤的概率。

$$Sel(A.C_a = B.C_b) = 1 / \max(NDV(A.C_a), NDV(B.C_b))$$

- **连接后行数：**连接后产生的期望行数为： $Rows(A \bowtie B) = Rows(A) \times Rows(B) \times Sel_1 \times Sel_2 \times \dots$ 其中 $Sel_1, Sel_2, \dots$ 是所有应用在节点 A 和 B 相关表之间的连接条件的选择率。如果没有连接条件（即笛卡尔积），则 $Rows(A \bowtie B) = Rows(A) \times Rows(B)$ 。
- **注意：**行数可以是浮点数，因为它代表的是一种数学期望。

2. 操作代价 (Cost Calculation)

扫描/过滤代价：在 SQL 执行时，会首先对所有 **FROM** 中的表（包含子查询产生的临时表）进行扫描，并在扫描过程中进行过滤。对于每个表 T_i ，其扫描/过滤代价为：

$$\begin{aligned} Cost(Filter(T_i)) = & Rows(T_i) \times 0.1 && \text{(基础 I/O 扫描代价)} \\ & + \sum_{\text{相关子链接}} Rows(T_i) \times Cost(\text{相关子链接}) && \text{(对每一行计算相关子链接)} \\ & + \sum_{\text{非相关子链接}} 1 \times Cost(\text{非相关子链接}) && \text{(非相关子链接只需计算一次)} \end{aligned}$$

不论 **WHERE** 中是否有与 T_i 表相关的过滤，均需要对其进行扫描。若 **WHERE** 中有该表相关的子链接，则需要额外考虑子链接的代价。

子链接的代价（即 $Cost(\text{相关子链接})$ 与 $Cost(\text{非相关子链接})$ 部分）= 按照子语句生成结果表的代价 + 对结果表进行聚合的代价。

例如，对于子链接条件 $T_1.c_1 = (\text{SELECT COUNT(*) FROM } T_2 \text{ WHERE } T_2.c_2 = T_1.c_3)$ ，其子链接的代价为 $Rows(T_2) \times 0.1 + Rows(T'_2) \times 0.05$ ，由两部分组成：

1. 生成结果表格，相当于执行 **SELECT * FROM T_2 WHERE $T_2.c_2 = T_1.c_3$** ，其代价为扫描表 T_2 的代价： $Rows(T_2) \times 0.1$
 2. **COUNT(*)** 聚合的代价为 $Rows(T'_2) \times 0.05$ ，其中 T'_2 表示过滤后的 T_2 ，即上一步中执行后的结果表；0.05 为聚合的代价系数。
- **相关条件的常量化**：由于相关子链接由外表驱动，在对子链接进行每次求值时， $T_1.c_3$ 的具体取值已确定，因此 $T_2.c_2 = T_1.c_3$ 被视为对 $T_2.c_2$ 的等值常量过滤。
 - **相关子链接**：由于外部列 $T_1.c_3$ 的值随 T_1 的每一行变化，需要对 T_1 的每一行重新计算整个子链接代价。
 - **非相关子链接**：子链接中的条件不依赖外部列，只需计算一次。
 - **子链接对行数的影响**：正如前面“行数估算”中所说，子链接条件 $T_1.c_1 = (\text{SELECT COUNT(*) ...})$ 被视为对列 c_1 的等值过滤，会将表 T_1 的行数按该列的 NDV 进行缩减： $Rows(T'_1) = Rows(T_1) \div NDV(T_1.c_1)$ （其中 T'_1 表示过滤后的 T_1 ）。
 - 在进行扫描时，会通过一次扫描处理完所有的过滤（包括子链接），因此公式中的 $Rows(T_i)$ 为扫描时行数。
 - 子链接内部的子查询如果包含更复杂的结构，需要递归应用本代价模型。

连接代价：我们支持两种连接算法。

- **哈希连接 (Hash Join)**：将表 A 的结果读入内存构建一个哈希表，然后流式读取表 B 的结果进行探测匹配。

$$Cost(HashJoin(A, B)) = Rows(B) \times 1.5 + Rows(A) \times 3.5 + Rows(A \bowtie B) \times 0.1$$

其中构建哈希表的代价为 $Rows(A) \times 3.5$ ，探测匹配的代价为 $Rows(B) \times 1.5$ ，结果扫描的代价为 $Rows(A \bowtie B) \times 0.1$ 。

- **嵌套循环连接 (Nested Loop Join)**: 对于表 A 中的每一行, 遍历表 B 中的所有行进行匹配。

$$Cost(NestedLoopJoin(A, B)) = Rows(A) \times Rows(B) \times 1.0 + Rows(A \bowtie B) \times 0.1$$

其中 $Rows(A) \times Rows(B) \times 1.0$ 是嵌套循环匹配的代价, $Rows(A \bowtie B) \times 0.1$ 是结果扫描的代价。

- **总代价 (Total Cost)**: 一个执行计划的总代价, 等于其中所有操作的代价之和。

3. 最优计划代价

最优计划: 通过不断调整 Join 的方式和顺序, 找到的 **总代价** 最低的执行计划。注意, 本题目中不能通过对 WHERE 中条件的逻辑推理进行优化。换句话说, 即使 WHERE $T.a=1, T.b=T.a, T.b=2$, 也需要按部就班去做计算。

提示: 先扫描表格并进行过滤, 再将过滤后的表进行连接。

任务要求

给定数据库中所有表的统计信息 (行数和每列的 NDV) 以及一个 BaseSQL 查询。你需要:

1. 解析 SQL, 理解查询意图。
2. 探索所有可能的执行计划 (即不同的连接顺序和连接算法组合)。
3. 使用上述代价模型计算每个计划的总代价。
4. 找出并输出那个总代价最小的 **最优执行计划**, 并给出其 **总代价** 和最终的 **估算行数**。

一个执行计划可以用括号表达式表示, 例如 $H(L(T2, T3), T1)$ 表示先用嵌套循环连接 T2 和 T3, 再将结果与 T1 进行哈希连接。

示例图解 对于计划 $H(L(T1, T2), T3)$, 其执行树如下所示:

图例:

- 叶子节点: 基础表或子查询
- 中间节点: 连接操作 (H= 哈希连接, L= 嵌套循环连接)
- 箭头方向: 自底向上执行

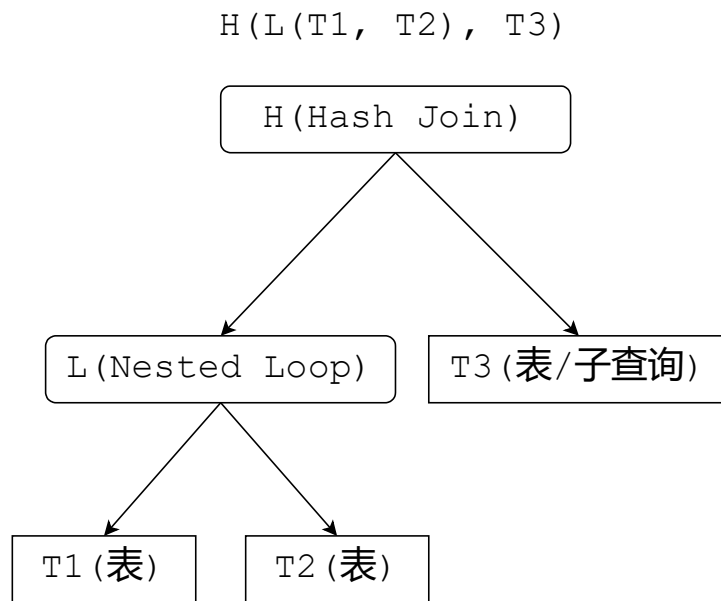
执行流程:

- T1 和 T2 通过嵌套循环连接 (Nested Loop Join) 生成中间结果
- 中间结果再与 T3 通过哈希连接 (Hash Join) 得到最终结果
- 该计划的 **Total Cost** 就是最终的执行代价值。

【输入格式】

从标准输入读入数据。

1. 第一行包含一个整数 N , 表示数据库中的基础表数量。
2. 接下来 N 个表的数据块, 每个块:
 - 第一行是表名 T_i , 一个整数 M 和 R , 表示该表有 M 列和 R 行。



- 接下来 M 行，每行是列名 C_j 和该列的 NDV 值。
3. 最后一部分是 SQL 查询，多行字符串，以分号 ; 结尾。SQL 规模限制：每层最多只有 12 个表，子链接和子查询的数量一共不超过 12。

【输出格式】

输出到标准输出。

1. 第一行输出两个浮点数，用空格隔开，分别表示最优计划的最终估算行数和总执行代价，保留两位小数。

【样例输入】

```

1 3
2 Students 3 10000
3 student_id 10000
4 name 9500
5 age 80
6 Courses 3 100
7 course_id 100
8 course_name 100
9 credits 10
10 Enrollments 2 100000
11 student_id 100
  
```

```
12 course_id 100
13 SELECT *
14 FROM (
15     SELECT *
16     FROM Students
17     WHERE Students.age = 21
18 ) AS SeniorStudents,
19 Enrollments,
20 Courses
21 WHERE SeniorStudents.student_id = Enrollments.student_id
22     AND Enrollments.course_id = Courses.course_id
23     AND Courses.credits = (
24         SELECT COUNT(*)
25         FROM Enrollments E2
26         WHERE E2.course_id = Courses.course_id
27     )
28     AND Courses.credits = (SELECT COUNT(*) FROM Enrollments);
```

【样例输出】

```
1 12.50 1133061.25
```

【样例解释】

- 节点 **SeniorStudents** (筛选 **Students** 表)
 - 行数 = $10000 \div 80 = 125$
 - 代价 (扫描) = $10000 \times 0.1 = 1000$
- 节点 **Enrollments** (基表扫描)
 - 行数 = 100000
 - 代价 = $100000 \times 0.1 = 10000$
- 节点 **Courses** (带相关 + 非相关子链接的筛选)
 - 基础行数 = 100
 - 子链接基数缩减: **credits** 列出现两次等值子链接 (一次相关、一次非相关)
 - 筛选后行数 = $100 \div 10 \div 10 = 1$ (**credits** 的 NDV = 10)
 - 代价 = $\text{cost}(\text{scan}) + \text{cost}(\text{correlated sublink}) + \text{cost}(\text{non-correlated sublink})$
 - $\text{cost}(\text{scan}) = 100 \times 0.1 = 10$
 - 相关子链接 (针对每个 **course_id** 一次):

- * 扫描 Enrollments: $100000 \times 0.1 = 10000$
- * 相关过滤后行数: $100000 \div 100 = 1000$
- * COUNT 聚合: $1000 \times 0.05 = 50$
- * 单次子链接代价: $10000 + 50 = 10050$
- * $\text{cost}(\text{correlated}) = 100 \times 10050 = 1,005,000$
- 非相关子链接 (仅一次):
 - * 扫描 Enrollments: $100000 \times 0.1 = 10000$
 - * COUNT 聚合: $100000 \times 0.05 = 5000$
 - * $\text{cost}(\text{non-correlated}) = 10000 + 5000 = 15000$
- 总代价 = $10 + 1,005,000 + 15000 = 1,020,010$

4. 执行计划对比分析。

枚举所有可能的连接顺序和连接算法组合后, 对比两个代表性方案:

方案 A (最优): $H(\text{SeniorStudents}, L(\text{Enrollments}, \text{Courses}))$

• 步骤 1: 嵌套循环连接 Enrollments 和 Courses

- 输入行数: 100000 (Enrollments)、1 (Courses 过滤后)
- 连接选择性: $1 \div \max(100, 100) = 1 \div 100$
- 输出行数: $100000 \times 1 \times (1 \div 100) = 1000$
- 代价: $\text{cost}(\text{Enrollments}) + \text{cost}(\text{Courses}) + \text{Rows}(\text{Enrollments}) \times \text{Rows}(\text{Courses}) \times 1.0 + \text{Rows}(\text{result}) \times 0.1$
- 代价: $10000 + 1020010 + 100000 \times 1.0 + 1000 \times 0.1 = 1130110$

• 步骤 2: 哈希连接 SeniorStudents 和中间结果

- 输入行数: 125 (SeniorStudents)、1000 (中间结果)
- 连接选择性: $1 \div \max(10000, 10000) = 1 \div 10000$
- 输出行数: $125 \times 1000 \times (1 \div 10000) = 12.5$
- 代价: $\text{cost}(\text{SeniorStudents}) + \text{cost}(\text{intermediate}) + \text{Rows}(\text{intermediate}) \times 1.5 + \text{Rows}(\text{SeniorStudents}) \times 3.5 + \text{Rows}(\text{result}) \times 0.1$
- 代价: $1000 + 1130110 + 1000 \times 1.5 + 125 \times 3.5 + 12.5 \times 0.1 = 1133061.25$

• 方案 A 总代价: 1133061.25

方案 B (次优): $H(H(\text{Courses}, \text{Enrollments}), \text{SeniorStudents})$

• 步骤 1: 哈希连接 Courses 和 Enrollments

- 输入行数: 1 (Courses 过滤后)、100000 (Enrollments)
- 连接选择性: $1 \div \max(100, 100) = 1 \div 100$
- 输出行数: $1 \times 100000 \times (1 \div 100) = 1000$
- 代价: $\text{cost}(\text{Courses}) + \text{cost}(\text{Enrollments}) + \text{Rows}(\text{Enrollments}) \times 1.5 + \text{Rows}(\text{Courses}) \times 3.5 + \text{Rows}(\text{result}) \times 0.1$

– 代价: $1020010 + 10000 + 100000 \times 1.5 + 1 \times 3.5 + 1000 \times 0.1 = 1180113.5$

• **步骤 2: 哈希连接中间结果和 SeniorStudents**

– 输入行数: 1000 (中间结果)、125 (SeniorStudents)

– 连接选择性: $1 \div \max(10000, 10000) = 1 \div 10000$

– 输出行数: $1000 \times 125 \times (1 \div 10000) = 12.5$

– 代价: $\text{cost}(\text{intermediate}) + \text{cost}(\text{SeniorStudents}) + \text{Rows}(\text{SeniorStudents}) \times 1.5 + \text{Rows}(\text{intermediate}) \times 3.5 + \text{Rows}(\text{result}) \times 0.1$

– 代价: $1180113.5 + 1000 + 125 \times 1.5 + 1000 \times 3.5 + 12.5 \times 0.1 = 1183052.25$

• **方案 B 总代价: 1183052.25**

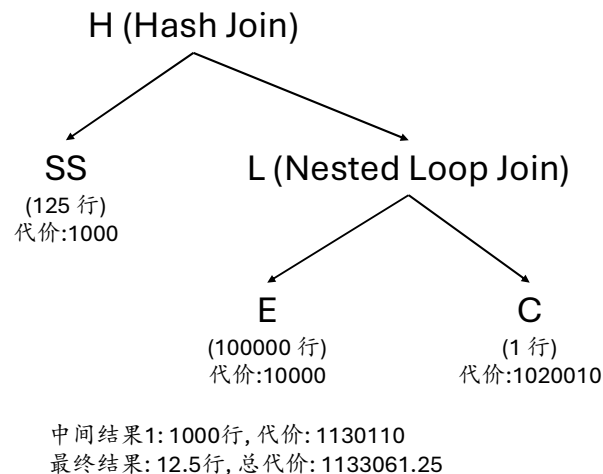
对比结论:

方案	执行计划	第一步代价	总代价	最终行数
A	H(SS, L(E, C))	1125110	1128048.75	12.5
B	H(H(C, E), SS)	1175113.5	1179802.25	

方案 A 在第一步对小表 **Courses** 使用嵌套循环连接, 避免了哈希构建的额外成本; 方案 B 第一阶段即构建哈希表, 导致成本高出约 52000。

最终答案: 估算行数 12.50, 总执行代价 1133061.25。

最优执行计划树 (方案 A):



其中: SS = SeniorStudents, E = Enrollments, C = Courses

说明:

- **子链接的基数影响 (可累积):** 子链接条件 $T.c = (\text{SELECT COUNT(*)} \dots)$ 视为对列 c 的等值过滤。若同一列出现多个等值条件 (例如同时存在相关与非相关子链接), 其基数缩减按 NDV 累乘; 本样例中 **Courses.credits** 同时受两条子

链接约束: $\text{Rows}(\text{Courses_filtered}) = 100 \div 10 \div 10 = 1$ (其中 `Courses_filtered` 表示过滤后的 `Courses` 表)。

- **计划空间探索**: 实际需要枚举所有可能的连接顺序 (包括不同的连接算法组合), 从所有执行计划中选择总代价最小的方案。
- **代价权衡**: 即使某个表过滤后很小, 但如果其扫描代价很高 (如包含相关子链接), 也不一定适合优先处理。

芯片系统设计优化 (chip)

【题目描述】

为了应对 AI 时代的计算挑战，公司设计了一款新型的 XPU 芯片，用于运行自研的 AI 模型。你需要帮助小新来完成这款 XPU 芯片设计优化工作。

芯片微架构

该芯片的微架构中总共有 5 个队列，分别为队列 A、队列 B、队列 C、队列 D、队列 E。其中队列 A 和队列 B 为计算队列，队列 C、队列 D、队列 E 为通信队列。自研 AI 模型的运行由一系列计算/通讯任务组成。这些任务已经被分配到了不同的计算/通讯队列：

- 队列 A: {任务 1 (A_task1), 任务 2 (A_task2), 任务 3 (A_task3)...}
- 队列 B: {任务 1 (B_task1), 任务 2 (B_task2), 任务 3 (B_task3)...}
- 队列 C: {任务 1 (C_task1), 任务 2 (C_task2), 任务 3 (C_task3)...}
- 队列 D: {任务 1 (D_task1), 任务 2 (D_task2), 任务 3 (D_task3)...}
- 队列 E: {任务 1 (E_task1), 任务 2 (E_task2), 任务 3 (E_task3)...}

除了队列之外，芯片上还有 5 个存储区域，分别为 **Buffer1**、**Buffer2**、**Buffer3**、**Buffer4** 和 **Buffer5**。这 5 个存储区域用于保存任务的输出结果。由于这 5 个存储区域在芯片之上，任务将输出结果保存在这些存储区域的时间可以忽略不计；任务从这些存储区域读取数据的时间也可以忽略不计。

任务也可以将输出结果保存在芯片外的公共内存（用“Out”表示）中，但这会带来额外的时间开销（不论数据大小，将输出结果保存到公共内存的时间为 100ns）。公共内存的空间可认为有无限大。

任务一：微架构的流水编排

你需要根据分配到不同队列的具体任务进行微架构的流水编排，使得 AI 模型能够尽可能快的完成执行。每个计算/通讯任务都有**执行时长**、**存储位置**和**使用资源的大小**。

- **执行时长**指任务从开始到结束所需要的时间，单位为 ns。
- **存储位置**为 Buffer1 到 5 之一，用于保存该任务的输出数据。编排时，也可以选择将该任务的输出保存在公共内存 Out 中，但这会额外增加 100ns 的执行时间。
- **使用资源大小**是指任务的输出数据大小，单位为 B。

任务的执行需要满足以下约束：

- 同一队列中的任务只能串行执行；不同队列中的任务可以并行执行；
- 任务之间的执行可能存在**依赖关系**，即后继任务依赖于前序任务的输出数据。例如：在队列 A 中，若 A_task2 任务依赖 A_task1 任务，则 A_task2 任务必须在 A_task1 任务完成后执行；不同队列间的任务也可能存在依赖。例如 B_task2 任务依赖 A_task1 任务，E_task3 任务依赖 B_task2 任务，则 B_task2 任务必须等 A_task1 任务执行完成之后才能开始执行，同理，E_task3 任务必须等 B_task2 任务执行完成之后才能开始执行。

- 若 A_task1 任务的输出数据保存在公共内存中, 后继任务 A_task2 和 B_task2 都依赖 A_task1 任务的输出数据, 则 A_task2 和 B_task2 各自执行时间均需要增加 100ns 用于从公共内存中读取数据。
- 当 B_task2 任务依赖 A_task1 任务, 且 A_task1 任务的输出为 0 时, A_task1 无输出, 因而 A_task1 没有额外的数据输出时间。
- 在同一时刻, 正在使用的资源总和不能超过对应资源的上限。例如, 同时执行的所有任务中, 对 Buffer1 的空间占用不应超过 Buffer1 的大小。虽然存储空间只用来保存输出数据, 但该空间在任务开始执行时即被占用。
- 当任务执行完成并且依赖它的所有其他任务也已执行完, 占用资源才可释放。例如: 队列 A 中任务 A_task1 正在执行, 占用资源 3072B(3kB), 执行完成后, 由于依赖它的 A_task2、B_task2 和 E_task2 任务均在执行, 所以资源暂时无法释放, 当这三个任务都执行完成后, 任务 A_task1 资源释放, 对应资源减少 3072B(3kB)。

在本任务中, 你将负责进行任务在微架构的流水编排, 也就是每个队列中各任务的执行顺序和时间。在输入文件中, 会给出该模型执行所涉及的各个队列中的任务及其属性 (*task_info*)、片上存储资源上限 (*memory*) 和任务依赖关系 (*dependency*)。调度器将会根据这些信息和约束条件, 给出每个队列里最佳的任务执行顺序, 使得模型图整体执行时间最短, 并且, 计算队列的时间利用率相对最高。

任务二：芯片性能与面积的权衡

假如芯片上的存储空间分配还可以调整, 为了在芯片性能和面积之间取得权衡, 需要在每个片上存储资源的存储大小上找到一个最佳值, 使得在最小成本 (面积) 下获得最佳的性能。

芯片的面积一般分为逻辑单元的面积和存储资源的面积, 在本题中假设逻辑单元已经固定, 你主要考虑对片上存储资源大小的设计来达成芯片性能与面积的平衡。其中片上存储资源的面积计算公式为:

$$\text{Area} = \max \left\{ 2 \text{ mm}^2, (\text{Size}_{\text{Buffer1}} + \text{Size}_{\text{Buffer2}}) \times (1.2 \text{ mm}^2/\text{Mbit}) \right. \\ \left. + (\text{Size}_{\text{Buffer3}} + \text{Size}_{\text{Buffer4}} + \text{Size}_{\text{Buffer5}}) \times (1.8 \text{ mm}^2/\text{Mbit}) \right\}.$$

其中, 总面积 Area 的单位是 mm^2 , $\text{Size}_{\text{Buffer1},2,3,4,5}$ 分别是各 Buffer 的存储容量大小。其中 $1\text{Mbit} = 1024 \times 1024 \text{ bits}$, $\text{Area} \geq 2 \text{ mm}^2$, 哪怕所有 Buffer 存储容量为 0。

在此任务中, 输入内容中的 *memory* 中的 5 个整数均为-1, 表示各类资源的大小是可以灵活调整的。你需要找到最佳的硬件资源大小配置, 使得推理任务执行时间更短, 同时资源大小最优。

【输入格式】

从标准输入读入数据。

输入的第一行为一个整数，表示测试点编号。此后包含三部分内容：

第一部分为各个任务的名称及其对应属性。此部分以一行 “--- task_info ---” 开头。每个任务占一行，分别其所在队列（单个字符，A 到 E）、任务名称（不含空格的字符串）、执行时间（整数）、资源位置（整数 1 到 5 对应 Buffer1 到 5）、使用资源大小（整数）。

例如：

```
1 --- task_info ---
2 A A_task1 2300 1 6144
3 B B_task2 1300 2 3072
```

说明名称为 A_task1 的任务在队列 A 中，执行时间是 2300ns，对应的存储资源是 Buffer1，使用资源大小是 6144B(6kB)。注意：这里是指任务 A_task1 的输出数据如果放在芯片存储区域中，则只能放在 Buffer1 中，但在也可以放在内存 Out 中。

第二部分为片上存储资源的容量。此部分以一行 “--- memory ---” 开头。之后一行为 5 个空格隔开的整数，分别对应 Buffer1 到 Buffer5 这五个存储资源的容量上限，单位为 B。

例如：

```
1 --- memory ---
2 65536 65536 262144 262144 65536
```

说明 Buffer1 资源上限是 65536B(64kB)，Buffer2 是 65536B(64kB)，Buffer3 是 262144B(256kB)，Buffer4 是 262144B(256kB)，Buffer5 是 65536B(64kB)。注意：任务可能使用的资源有 Buffer1、Buffer2、Buffer3、Buffer4、Buffer5 和 Out，但 Out 在输入数据中不给出，在任务 2 中也不纳入面积计算公式。

第三部分为任务之间的依赖关系。此部分以一行 “--- dependency ---” 开头。此后每一行包含一对空格隔开的任务名，表示后者任务依赖于前者任务。例如：

```
1 --- dependency ---
2 A_task1 A_task2
3 A_task1 B_task2
4 B_task2 E_task3
```

说明 A_task2 任务依赖 A_task1 任务，B_task2 任务依赖 A_task1 任务，E_task3 任务依赖 B_task2 任务。

【输出格式】

输出到标准输出。

输出结果应包含 3 方面内容：

1. task_order: 任务编排方案。在 “--- task_order ---” 行后, 应该包含五行输出, 分别对应队列 A 到队列 E 中的任务序列。每一行包含由 “,” 隔开的所有任务名称, 顺序与执行顺序严格对应。
2. task_info: 每个任务及其具体信息。在 “--- task_info ---” 行后, 有若干行。每行表示一个任务的执行信息。任务之间的顺序与其在输入中的顺序相同。每行首先打印任务的名称和一个 “:”, 之后为空格隔开的 4 个整数, 分别表示任务开始时间、任务实际执行时间 (如果有涉及到 Out 中读取数据和输出数据, 需要包含在此时间中)、片上存储资源占用量 (单位: B)、Out 空间占用量 (单位: B)。两个占用量中一定有一个为 0。
3. hardware_info: 硬件资源信息。在 “--- task_info ---” 行后, 有一行五个空格隔开的整数。按顺序分别是 Buffer1、Buffer2、Buffer3、Buffer4 和 Buffer5 这五类资源。对于任务 1 可以沿用原有输入; 对于任务 2 必须输出不受约束下的最佳资源配置 (单位: B)。

格式示例如下:

```
1 --- task_order ---
2 A_task1, A_task3, A_task2
3 B_task1, B_task2, B_task3
4 C_task1, C_task2, C_task3
5 D_task1, D_task2, D_task3
6 E_task1, E_task2, E_task3
7 --- task_info ---
8 A_task1: 0 2300 6144 0
9 B_task2: 2300 1300 0 3072
10 --- hardware_info ---
11 65536 65536 262144 262144 65536
```

【样例输入】

```
1 0
2 --- task_info ---
3 B B_task0 55 1 12288
4 B B_task1 16 1 3072
5 B B_task10 6 1 256
6 B B_task11 27 1 3
7 B B_task12 12 1 256
8 B B_task13 6 1 256
9 B B_task2 46 1 256
```

```
10 B B_task3 8 1 768
11 B B_task4 55 1 12288
12 B B_task5 101 1 12288
13 B B_task6 85 1 12288
14 B B_task7 27 1 6144
15 B B_task8 11 1 1536
16 B B_task9 48 1 256
17 D D_task0 1286 1 12288
18 --- memory ---
19 262144 65536 65536 262144 1048576
20 --- dependency ---
21 B_task10 B_task11
22 B_task4 B_task6
23 B_task3 B_task10
24 B_task11 B_task9
25 B_task9 B_task12
26 B_task9 B_task2
27 B_task5 B_task7
28 B_task7 B_task12
29 B_task10 B_task9
30 B_task3 B_task13
31 D_task0 B_task4
32 D_task0 B_task0
33 B_task5 B_task12
34 B_task1 B_task8
35 B_task12 B_task2
36 B_task13 B_task10
37 B_task0 B_task5
38 B_task8 B_task3
39 B_task0 B_task4
40 B_task7 B_task1
```

【样例输出】

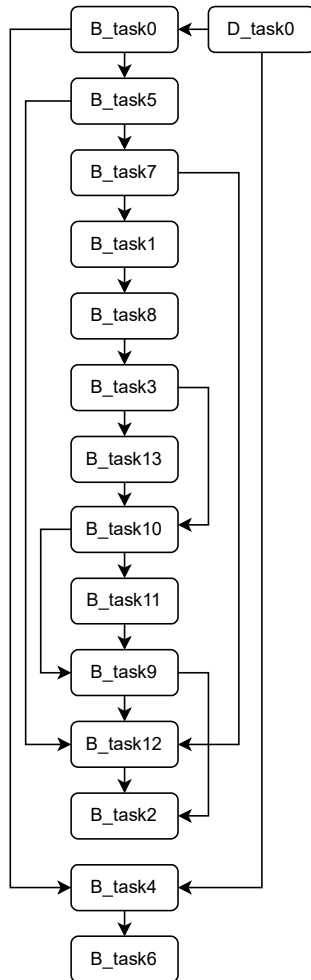
```
1 --- task_order ---
2
3 B_task0, B_task5, B_task7, B_task1, B_task8, B_task3, B_task13,
```

```

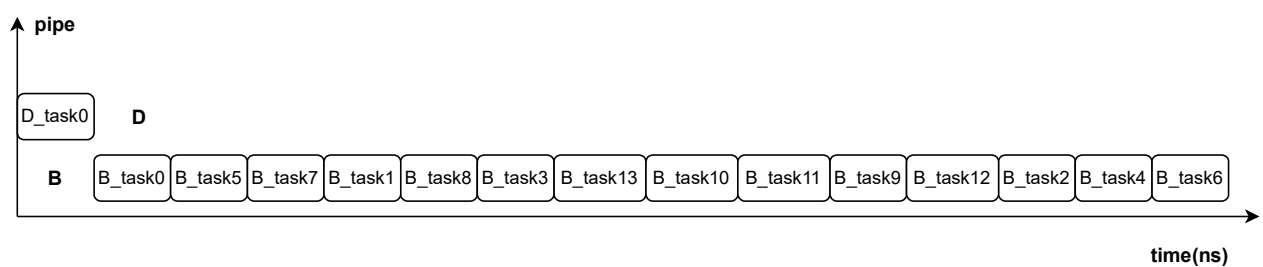
4      B_task10, B_task11, B_task9, B_task12, B_task2, B_task4, B_task6
5  D_task0
6
7  --- task_info ---
8  B_task0: 1286 55 12288 0
9  B_task1: 1471 16 3072 0
10 B_task10: 1514 6 256 0
11 B_task11: 1520 27 3 0
12 B_task12: 1595 12 256 0
13 B_task13: 1507 6 256 0
14 B_task2: 1607 46 256 0
15 B_task3: 1499 8 768 0
16 B_task4: 1657 55 12288 0
17 B_task5: 1342 101 12288 0
18 B_task6: 1712 85 12288 0
19 B_task7: 1443 27 6144 0
20 B_task8: 1488 11 1536 0
21 B_task9: 1547 48 256 0
22 D_task0: 0 1286 12288 0
23 --- hardware_info ---
24 262144 65536 65536 262144 1048576
```

【样例解释】

在样例中，各个 task 之间的依赖关系如下图所示。此样例中只包含了 B 和 D 两个队列。



根据已知的依赖关系，在重排流水线时，D_task0 任务应首先执行，完成之后再执行 B_task0 任务，之后再依次执行其他任务。（样例为简单化，没有列举出不同队列间并行执行任务的情况，但实际用例中存在），如下图所示：



根据重排后的流水线执行顺序，可以分别计算出各个 Buffer 所需的内存大小，进而计算出面积。

【评分规则】

任务 1 的评分规则：

时间基线是 100000 ns，你求解结果的整体执行时间是 T_{total} (ns)，队列 A 和队列

B 的时间分别是 T_A 和 T_B (ns)，评分通过以下公式计算：

$$Score_{\text{Perf}} = \frac{100000}{T_{\text{total}}} \times 80\% + \frac{(T_A + T_B)}{T_{\text{total}}} \times 20\%$$

注意，只有在通过正确性检测之后，才能参与排名拿到分数。

为保证通过正确性测试，按照题目要求，并保证：

1. 输出文件中的内容格式应符合要求；
2. 每个队列需要执行的任务数量在重排前后应该相同；
3. 同一队列内的任务必须能串行执行；
4. 有依赖关系的两个任务必须能正确执行；
5. 同一时刻，片上资源的使用大小不能超出资源上限；
6. 若某任务使用了内存 Out，则该任务的实际执行时间必须考虑内存读写时间。
7. 答案中所有的数字均为非负整数，同时不超过 10^{12} 。

任务 2 的评分规则：

$$Score_{\text{PA}} = \frac{1000 \times Score_{\text{Perf}}}{\text{Area}}$$

其中 $Score_{\text{Perf}}$ 为任务 1 中的性能评价公式，值越高表明性能越好；**Area** 则是存储资源的面积，值越低越好。

每个测试点的分数：

对于任务一的测试点，得分为：

$$Score = \text{BaseScore} + S_{\log}(Score_{\text{Perf}})$$

对于任务二的测试点，得分为：

$$Score = \text{BaseScore} + S_{\log}(Score_{\text{PA}})$$

其中 **BaseScore** 为通过正确性校验后的基础得分，每个测试点 5 分。其中 $S_{\log}$ 为标准排名得分函数，定义如下：

$$S_{\log} = \text{BaseRankScore} \times \frac{\log(N+1) - \log(R)}{\log(N+1)}$$

其中 **BaseRankScore** 为通过排名部分总分，每个测试点 5 分， N 为在该测试点通过正确性校验的参赛选手总人数， R 为该测试点中你的排名（1 表示最好）。

题目总分数：

本题目以 **最后一次** 提交的程序分数为准，总分数为所有测试点得分之和。

为了方便大家进行调试，我们提供了所有测试点的输入文件，同时提供了一个验证程序用于检查输出文件的正确性。验证程序接收两个参数，分别为输入文件的路径和答案文件的路径，使用方法如下：

```
1 ./verifier <input_file> <output_file>
```

验证通过会打印 **Verification passed!** 和一些统计信息。

```
1 Verification passed!  
2 Total execution time: 1076543 ns  
3 Queue A execution time: 31048 ns  
4 Queue B execution time: 1054324 ns  
5 Area: 21.9 mm^2  
6 Score_Perf: 0.275952
```